

Assignment 1/2026 — Bangkok Transit Navigator: Web App Edition

010153103 Algorithms and Data Structures

Semester 1 / Academic Year 2026

Department of Electrical and Computer Engineering (ECE)

Faculty of Engineering, King Mongkut's University of Technology North Bangkok

This assignment builds on the graph and shortest-path concepts from Lectures 18–19 (Graphs, Shortest Paths). If you have not yet reviewed those lectures, do that first — this assignment assumes you already know the Graph ADT, Dijkstra's algorithm, and priority queues.

1 Scenario

You will build a small web application, the **Bangkok Transit Navigator**, that lets a rider:

1. Enter an **origin** and **destination** station.
2. Enter either a **departure time** ("I'm leaving at 08:15, when do I arrive?") or a **desired arrival time** ("I need to be there by 09:00, when should I leave?").
3. Choose how the route should be optimized: **fastest total time**, or **fewest interchanges**.
4. See the resulting itinerary as both text (stop-by-stop) and a **line-colored map** with the chosen route highlighted.

This is a full application with a graph engine and a browser-based UI, and it must reason about *time*, not just hop count.

2 Learning Objectives

- Build a weighted graph from a real-world, imperfectly-structured CSV file.
- Model a genuinely tricky real-world constraint (line-transfer penalties) as a graph modeling problem, not just an algorithm problem.
- Implement Dijkstra's algorithm with **two different cost functions** (time vs. interchange count) against the same underlying graph.
- Render a graph visually, using line identity (color) as the primary visual encoding.
- Design a small, usable UI on top of a correct backend algorithm.

3 Provided Data

File: `assignment/bangkok_transit/bangkok_network_graph.csv`

Column	Meaning
Source_Station_Code	Unique code for the source <i>platform</i> (e.g. N8, BL12)
Source_Station_Name	Human-readable name (e.g. Mo Chit)
Target_Station_Code	Unique code for the target platform
Target_Station_Name	Human-readable name
Edge_Type	Line Track (ride between adjacent stations on one line) or Interchange Connection (walk between platforms to change lines)
Line_Context	Which line the edge belongs to, e.g. BTS Sukhumvit Line , MRT Blue Line , or a description of which two lines an interchange connects

Read the whole file before you start coding. It has a few properties that will matter for your design:

- **Station codes are the real identity, not names.** Several physical interchange stations are represented by *two different codes* for the same location — e.g. Bang Wa is S12 (BTS Silom) and BL33 (MRT Blue); Sukhumvit is E4 (BTS) and BL21 (MRT); Tao Poon is BL09 (MRT Blue) and PP16 (MRT Purple). Build your graph keyed on *code*, and only use the name for display.
- **Line Track rows are listed once per direction** as the line was traced, but riders can travel either way along the track. Decide how you will represent that (e.g. treat the graph as undirected, or insert both directions when you parse each row) and be consistent.
- **The MRT Blue Line is not a simple line** — it has a loop and a branch. Trace it on paper before you code: BL01 (Tha Phra) closes a loop back to itself via BL31, and also branches off towards BL37 (Lak Song). Your graph representation must support this without special-casing it.
- **The dataset has a minor duplicate.** The edge between BL01 and BL32 appears twice, once tagged MRT Blue Line and once MRT Blue Line Branch. This is a realistic data-quality quirk, not a bug in the assignment — decide whether to de-duplicate it and say so in your report.
- **One row is a self-loop:** CEN,Siam,CEN,Siam,Interchange Connection,Cross-Platform Interchange (Sukhumvit <-> Silom). Read section 6.1 before deciding how to handle it — it is the single most important modeling decision in this assignment.

4 Part 1 — Build the Graph

- Parse the CSV into an in-memory graph using the Graph ADT style from Lecture 18 (vertices and edges as objects, adjacency list keyed by vertex).
- Vertices are station *codes*; store the display name and line as vertex attributes.
- In your report, justify your choice of adjacency list vs. adjacency matrix in terms of the network’s sparsity (count the vertices and edges and say so explicitly).

5 Part 2 — Visualize the Network

Render every station and every Line Track edge, with **each line drawn in a distinct, consistent color**:

Line	Suggested color	
BTS Sukhumvit Line		Light green
BTS Silom Line		Dark green
MRT Blue Line (incl. branch)		Blue
MRT Purple Line		Purple (only the PP16 connection point appears in the data)
Interchange Connection edges		Neutral grey, dashed — reads as “walk here,” not “ride here”

You do **not** need real geographic coordinates. A clean, readable layout (e.g. laying each line out as a path/loop and placing interchange edges as short connectors between them) is perfectly acceptable — the grading criterion is that a viewer can tell, at a glance, which line is which and where the transfer points are. Client-side visualization libraries (D3.js, Cytoscape.js, vis-network, or even hand-rolled SVG/Canvas) are all fine here — see the constraints in section 8. The graph-building and pathfinding code must be your own.

5.1 Highlighting a computed route

Once Part 4 is working, the same visualization must be able to highlight a computed itinerary on top of the full map (e.g. thicker/bolder stroke on the traveled edges, and markers on the boarding/alighting/interchange stations).

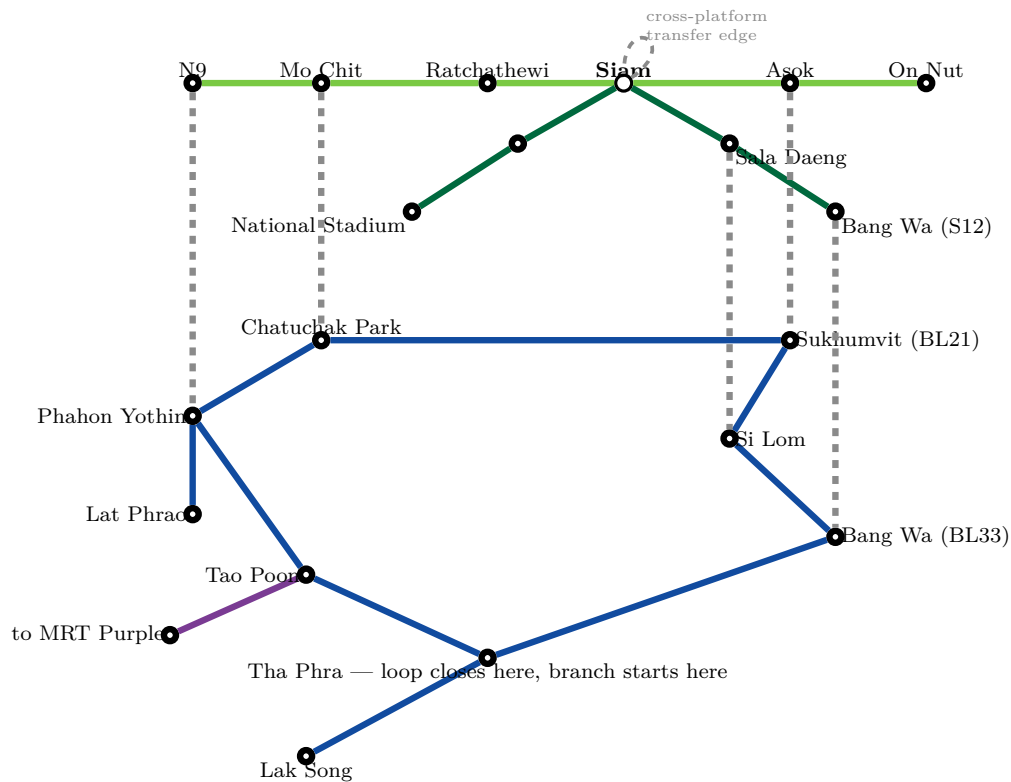


Figure 1: Simplified schematic of the network (not all 60+ stations shown; line colors match the table above), illustrating the Blue Line loop and its branch to Lak Song, ordinary interchange links (dashed grey) between distinct station codes, and the Siam cross-platform self-loop (dashed loop at CEN). Your submission’s map should cover the full CSV.

6 Part 3 — Weighting the Graph

You must assign a **time cost** to every edge:

- **Line Track edges:** the CSV has no travel-time column, so you must choose and justify a constant per-segment time. A reasonable default is **2.5–3 minutes** between adjacent stations (typical BTS/MRT scheduled time) — you may use a different number if you justify it.
- **Interchange Connection edges between two different station codes** (e.g. E4 ↔ BL21, S2 ↔ BL25, N8 ↔ BL12, N9 ↔ BL13, S12 ↔ BL33, BL09 ↔ PP16): these represent a real walk between separate platforms/buildings. A reasonable default penalty is **5–7 minutes**.

6.1 The Siam gotcha — read this before you code

The Siam (CEN) Gotcha

At Siam, both the Sukhumvit and Silom lines use the **same vertex code**, CEN. That means a naive graph walk from, say, N1 (Ratchathewi) to S1 (Ratchadamri) can pass straight through CEN — arriving on one line and leaving on the other — **without ever being forced to use the CEN→CEN self-loop edge**, because CEN already connects directly to both N1 and S1. If your shortest-path algorithm only tracks “which station am I at,” it will silently give this transfer a cost of zero, which is wrong: a rider here really does have to cross to the opposite platform.

You need a strategy that reliably charges the cross-platform penalty whenever a path’s `Line_Context` changes at CEN, and — for consistency — you should think about whether the same strategy also correctly counts an interchange for the fewest-interchanges optimization mode (section 7.3). Two directions worth considering:

- Track **which line the rider is currently on** as part of your search state (e.g. search over `(station_code, current_line)` pairs instead of bare station codes), so that arriving at CEN on the Sukhumvit line and leaving on the Silom line is a detectable, chargeable transition — the same technique naturally also covers the explicit **Interchange Connection** edges elsewhere in the network.
 - Or, keep the simpler bare-station-code graph and add explicit bookkeeping in your path-reconstruction step that specifically detects a **Line_Context** change at CEN.
- Either is acceptable. What is **not** acceptable is a route through Siam that silently costs nothing to change lines. Explain and justify whichever approach you took in your report.

7 Part 4 — The Web App

7.1 Departure mode

Given an origin, destination, and a departure time (HH:MM), compute the shortest-cost path and report:

- Estimated **arrival time** = departure time + total travel time along the path.
- The full itinerary (see 7.4).

7.2 Arrival mode

Given an origin, destination, and a **desired arrival time**, compute the same shortest path and report the **latest departure time** that still meets it: $\text{departure} = \text{arrival} - \text{total travel time}$.

Note

Because this dataset has no train schedule/frequency information, edge weights don't depend on time of day, so “plan by arrival” is intentionally the mirror image of “plan by departure” rather than a separate search. If you want a harder version of this, see the bonus in section 12.

7.3 Optimization mode

The user must be able to choose between two objectives, computed against the **same graph**:

- **Fastest route:** minimize total time (as weighted in section 6).
- **Fewest interchanges:** minimize the number of line changes; use total time only to break ties between routes with the same number of interchanges.

Implement this as two different cost functions/comparators feeding the same Dijkstra implementation (or two clearly related variants) — don't just hard code two unrelated algorithms. Your report should explain how you adapted Dijkstra (or, if you chose it, a different single-source shortest-path approach) for the fewest-interchanges objective, and how you're counting an interchange consistently with your answer to section 6.1.

7.4 UI requirements

- Origin/destination pickers (autocomplete or dropdown over station names — remember several names map to more than one code; if the user picks an interchange station by name, either ask which platform or treat it as “already there,” whichever you justify).
- A time input, and a toggle between “Depart at” / “Arrive by.”
- A toggle between “Fastest” / “Fewest interchanges.”
- Output: a stop-by-stop itinerary — e.g. “Board BTS Sukhumvit Line at Mo Chit (08:15) → ride to Siam (08:32) → cross platform to BTS Silom Line (08:35) → ride to Sala Daeng (08:44). Total: 29 min, 1 interchange.” — plus the highlighted map from Figure 1.

- Basic error handling: identical origin/destination, unknown station, malformed time.

Figure 2: Illustrative wireframe of the required UI elements: station pickers, departure/arrival toggle, optimization toggle, a highlighted map view, and a stop-by-stop itinerary with total time and interchange count. Visual styling is entirely up to you — only the elements shown are required.

8 Technical Constraints

- **You must implement the graph, Dijkstra’s algorithm (or your chosen shortest-path algorithm), and the priority queue yourself.** Do not call a library’s built-in shortest-path or graph-routing function — that is the part of this assignment being assessed.
- Any language/stack is fine (e.g. vanilla JavaScript/TypeScript in the browser; a small Node/Express, Java, or Python backend with an HTML/JS frontend). Pick whichever lets you finish; a simple single-page client-side app is entirely sufficient.
- For **rendering only** (drawing the map, laying out nodes, UI widgets), third-party libraries are fine — that part isn’t the DSA content of this assignment.

9 Deliverables

- **Source code**, organized and readable, with instructions to run it locally (e.g. README.md with setup steps).
- **Report** covering: your name and student ID; graph representation choice and sparsity justification; how you modeled and charged interchange penalties, including your answer to the Siam (CEN) gotcha; how you adapted Dijkstra for the fewest-interchanges objective; the travel-time and interchange-penalty constants you chose and why; and any data-cleaning decisions (e.g. the BL01↔BL32 duplicate).
- **Example runs** (screenshots or text) for the test cases in section 11.

10 Grading Rubric

Criterion	Weight
Graph correctly built from the CSV (handles codes vs. names, the loop/branch, direction)	20%
Correct time-weighted shortest path, both departure and arrival modes	20%
Correct fewest-interchanges mode, including the Siam case	20%
Visualization: lines colored and distinguishable, route highlighting works	15%
Report: clear justification of design decisions above	15%
UI usability & error handling	10%

11 Example Test Cases

Verify your app against at least these routes (exact times will depend on your chosen constants — what’s graded is that the logic and interchange count are correct):

1. **Bang Wa (S12/BL33) → Lat Phrao (BL14)** — exercises the BTS↔MRT interchange at Bang Wa and the MRT Blue Line loop/branch.
2. **Lak Song (BL37) → Kheha (E23)** — a long cross-network trip, traversing most of the Blue Line then most of the Sukhumvit Line.
3. **Ratchathewi (N1) → National Stadium (W1)** — the Siam gotcha: this route must reflect a cross-platform interchange cost at CEN, not a free pass-through.
4. Any pair of your choosing, once in “**fastest**” mode and once in “**fewest interchanges**” mode, where the two modes give a *different* route — to demonstrate the modes are genuinely independent.

12 Bonus Challenges (optional, extra credit)

- **Train frequency / headway:** instead of instant transfers, assume each line runs every N minutes and add an expected wait time when boarding or transferring — this makes “plan by arrival” genuinely asymmetric with “plan by departure,” unlike the base assignment.
- **First/last train:** model service hours and reject or adjust queries outside them.
- **Station closure:** let the user mark a station “closed for maintenance” and recompute the best detour.
- **Shareable route:** encode a computed trip in the URL so it can be copied and reopened directly to that result.

13 AI Collaboration

You may use an AI assistant to discuss graph-modeling approaches (e.g. “how do I represent a rider’s current line as part of a Dijkstra search state?”) or to debug. You may not have it design your whole solution for you, and you must be able to explain, live, any part of your submitted code — this includes your justification for how you handled the Siam interchange case. Disclose any significant AI assistance in your report.

14 Submission

Submit your source code repository link (or archive) and report PDF via the course site by the announced deadline.

This assignment is subject to minor adjustments at the instructor's discretion. Students are responsible for staying informed of any changes announced in class or via the course website.